
Synthetic Pretraining for Single-Object Detection in Underwater Environments

Jacob Helzner

Department of Computer Engineering
Stony Brook University
Stony Brook, NY 11790
jacob.helzner@stonybrook.edu

Abstract

Deep ocean plastic remains a serious ecological and climate risk. Though Autonomous Underwater Vehicles (AUV's) show promise in cleanup efforts, the computer vision systems they rely on struggle in the turbid, low light environment of the deep ocean. Furthermore, while large public datasets exist for training - such as TrashCan and FathomNet - annotated images of deep sea plastic trash remain scarce. This paper advances pretraining on domain-randomized synthetic data as a potential means of learning useful priors before working with limited real-world data. We accomplish this by computing the mAP (mean average precision) across varied finetuning runs of YOLO, a foundational bounding box model, paying close attention to the effect of pretraining on performance. As an illustrative example, we focus on the single object class of plastic bags. Initial evidence invites further research into validity of the synthetic-prewarming approach for deep-sea object detection.

1 Introduction

Deep sea plastic poses a significant risk to ecological stability and may also play a role in oceanic Carbon Dioxide absorption. One 2025 study in Nature magazine highlights the effect of deep sea plastic decomposition - as plastics degrade, they may disrupt the system by which plankton and marine organisms move carbon from surface water to the deep ocean, thereby reducing the rate at which the ocean can absorb Carbon Dioxide in the atmosphere. The study also showed that the fraction of total particulate carbon increased from 0.1 percent at 30m to 5 percent at 2000m, further underscoring the need to remove accumulated plastic from seabeds, in conjunction with dumping remediation to make these efforts meaningful.

Since the mid 2010's, monumental advances have been made in the field of computer vision, particularly with respect to CNN's and publicly available segmentation models such as YOLO which compute bounding boxes over an image in a single pass. In conjunction with recent advances in LLM-informed robotic task and motion planning (TAMP), as well as recent research in domain randomization for RL policy learning in simulation, this has opened unique opportunities for deep ocean plastic cleanup, such as underwater robotic manipulation by Autonomous Underwater Vehicles (AUVs). The detection of underwater trash is also useful to ocean monitoring efforts, which help researchers determine targeted strategies for environmentally-motivated policy.

Some major benefits of underwater object detection include fast real time processing and the ability to locally install detection systems on underwater equipment. These features are critical in deep sea environments, where relying on high-bandwidth communication to the surface is infeasible. Furthermore, a carefully trained vision model shows promise in reducing the number of "false

positives" detected in older methods such as sonar-based mapping, thereby reducing the accidental targeting or destruction of wildlife.

Despite this, deep sea object detection continues to face significant challenges, most of which are common-sense knowledge about the deep sea environment. The first of these is optical noise and low ambient light conditions. Suspended particles (also known as turbidity) scatter artificial light sources, creating a strong blur that obscures object boundaries. Additional semantic information is lost to color distortion - with increasing depth, red light decreases faster than blue or green, making colored plastic materials harder to spot. Another major challenge to deep sea trash recognition is the lack of data available. While large datasets like TrashCan exist, true deep sea imagery is somewhat scarce due to the technical difficulty of reaching the deep ocean. The data available in TrashCan specifically is used in this paper to train and finetune the model to be discussed shortly. One potential solution to object detection in the limited-data regime borrows from robotics research - namely, pretraining on simulated data with heavy domain randomization. This paper will experimentally assess the merits of such an approach, gathering insights on how finetuned vision models can most effectively learn useful priors before working with limited real-world data.

2 Problem Statement

As an illustrative example, the goal is to find the bounding box of a plastic bag from a deep sea photo. When targeting a small, closed set of recurring objects like bags and cans, using single-object detectors may become advantageous. While somewhat outside of the scope of this paper, it's worth mentioning that the low total number of classes may make a 'Mixture of Experts' detector practical due to the low overhead of a few models running in parallel. This approach could tighten the decision boundary for each individual class by decoupling their feature spaces, ultimately reducing false positives.

Stated formally, the problem of bounding box localization is as follows:

Let the RGB image be a 3D tensor $I \in \mathbb{R}^{H \times W \times 3}$. For a single object detector, we define the parameterized model, f_θ , as follows:

$$f_\theta(I) = (\hat{p}, \hat{B})$$

Where $\hat{p} \in [0, 1]$ is the predicted confidence that the target object is present in the image, and $\hat{B} \in \mathbb{R}^4$ is the predicted bounding box geometry within the image. During training, the dataset provides a ground truth label for each image I_i , defined as $Y_i^* = (p_i^*, B_i^*)$. If $p_i^* = 0$, B_i^* is undefined. From here, we find the optimal parameters θ^* that minimize a joint loss function over a dataset of N images. The total loss $\mathcal{L}_{total}$ is a weighted sum of the classification and localization losses:

$$\mathcal{L}_{total}(\theta) = \frac{1}{N} \sum_{i=1}^N \left[\mathcal{L}_{cls}(\hat{p}_i, p_i^*) + \lambda \cdot p_i^* \cdot \mathcal{L}_{loc}(\hat{B}_i, B_i^*) \right]$$

To ensure localization loss is not calculated for examples where the object is not present, we multiply $\mathcal{L}_{loc}$ by p_i^* . Additionally, we introduce a scaling parameter, λ , to ensure the bounding box loss doesn't dominate the classification loss, or vice versa - λ can be determined through a cross validation strategy such as grid search.

We now discuss each component of the loss. Since the classification task is strictly binary, we can model $\mathcal{L}_{cls}$ using Binary Cross-Entropy:

$$\mathcal{L}_{cls}(\hat{p}, p^*) = -(p^* \log(\hat{p}) + (1 - p^*) \log(1 - \hat{p}))$$

For the localization loss $\mathcal{L}_{loc}$, we have a few options. While more sophisticated metrics exist - which we will soon briefly mention - a standard distance-based approach is:

$$\mathcal{L}_{loc}(\hat{B}, B^*) = ||\hat{B} - B^*||$$

Lastly, we must define how the trained model makes an inference. Given the output for a novel photo, $(\hat{p}, \hat{B})$, a threshold-based decision rule is:

$$\text{Detection} = \begin{cases} \hat{B} & \text{if } \hat{p} \geq \tau \\ \text{Null} & \text{if } \hat{p} < \tau \end{cases}$$

Where $\tau \in [0, 1]$ is a confidence threshold. Similarly for λ , this value can be tuned via cross validation.

3 A Note on Technical Implementation

While we have just covered the single-object expert using a standard spatial loss metric, the actual implementation in this paper relies on the fine-tuning of a YOLO architecture. As a result, the actual, optimized loss function is slightly different. Specifically, YOLO minimizes $\mathcal{L}_{YOLO} = \lambda_{cls}\mathcal{L}_{cls} + \lambda_{box}\mathcal{L}_{CIoU} + \lambda_{dfl}\mathcal{L}_{dfl}$, where bounding box error is handled by Complete Intersection over Union (CIoU) to ensure consistency across aspect ratios, and Distribution Focal Loss (DFL) to model boundary ambiguity. Since the technical rigor of the YOLO architecture is out of the scope of this paper, we briefly acknowledge the differences from the vanilla theoretical approach.

It’s also worth mentioning that the concept of synthetic prewarming generalizes across visual learning models - as a result, the performance improvements observed in this paper validate the usefulness of synthetic data for deep-sea visual tasks, regardless of the specific framework.

4 Constructing the Synthetic Dataset

We now outline the approach taken to replicate deep sea imagery. The principle of domain randomization ensures robustness to accidentally overfitting the simulation during training - in other words, a loose approximation of the target environment is often enough to capture meaningful semantic information. That being said, background in marine ecology would likely be useful to create a targeted simulation (for example ocean floor vs trench environment).

All synthetic images were rendered in Blender 4.4.3, with additional post processing implemented via direct pixel manipulation in Python’s PIL library. For the rendering step, the Blender simulation used a total of 56 randomized parameters, including random noise texture parameters, object generation and scatter, as well as lighting and color settings. The plastic bag itself was implemented as a deforming mesh using Blender’s native cloth simulation system. Since the form of a plastic bag is greatly affected by the surrounding water flow, parameters such as turbulence and wind strength were used to perform a fluid simulation on the bag object, ensuring rough physical accuracy. For completeness, the parameters and their settings are listed in the table below. We also include a sample of the render output (See Figure 1).

Note: A few of these parameters, such as LIGHTING BOOST and DEBRIS USE RIGIDBODIES, are technical workarounds to overcome the idiosyncrasies of Blender’s rendering system. Since a complete discussion of the rendering system with respect to this project is beyond the scope of this paper, we leave their interpretation to the reader.

Parameter	Value
Cloth Physics	
BAG_MASS	0.15
BAG_BENDING_STIFFNESS	0.0003
BAG_TENSION_STIFFNESS	0.08
BAG_COMPRESSION_STIFFNESS	0.03
BAG_SHEAR_STIFFNESS	0.3
BAG_AIR_DAMPING	1.1
BAG_BENDING_DAMPING	0.35
BAG_CLOTH_QUALITY	3
BAG_COLLISION_QUALITY	3
Environment	
GRAVITY	(0, 0, -20.0)
TURBULENCE_STRENGTH	35.0
TURBULENCE_SIZE	3.0
TURBULENCE_FLOW	1.0
WIND_STRENGTH	10.0
WIND_NOISE	3.0

Continued on next page

Continued from previous page

Parameter	Value
Lighting	
WORLD_COLOR_RANGE	((0.02, 0.06, 0.08), (0.05, 0.12, 0.14))
WORLD_STRENGTH_RANGE	(2.0, 6.0)
FOG_DENSITY_RANGE	(0.002, 0.01)
USE_FOG	False
SUN_STRENGTH_RANGE	(0.6, 2.0)
SUN_COLOR_RANGE	((0.2, 0.35, 0.4), (0.35, 0.5, 0.55))
FILL_STRENGTH_RANGE	(20.0, 80.0)
EXPOSURE_RANGE	(0.03, 0.2)
LIGHTING_BOOST	3.0
LIGHT_BRIGHTNESS_RANGE	(0.03, 0.2)
SUN_ANGLE_RANGE	(0.02, 0.25)
AREA_SIZE_RANGE	(6.0, 14.0)
Materials	
FLOOR_BASE_COLOR_RANGE	((0.06, 0.08, 0.07), (0.14, 0.18, 0.12))
FLOOR_ROUGHNESS_RANGE	(0.7, 0.95)
FLOOR_NOISE_SCALE_RANGE	(2.0, 6.0)
DEBRIS_BASE_COLOR_RANGE	((0.08, 0.1, 0.1), (0.2, 0.2, 0.18))
DEBRIS_ROUGHNESS_RANGE	(0.6, 0.9)
BAG_BASE_COLOR_RANGE	((0.12, 0.2, 0.22), (0.35, 0.45, 0.48))
BAG_DARK_COLOR_RANGE	((0.03, 0.06, 0.07), (0.12, 0.16, 0.18))
BAG_ALPHA_RANGE	(0.15, 0.45)
BAG_ROUGHNESS_RANGE	(0.65, 0.95)
MATTE_SPECULAR_RANGE	(0.0, 0.15)
BAG_TRANSMISSION_RANGE	(0.0, 0.05)
BAG_NOISE_SCALE_RANGE	(2.0, 8.0)
BAG_NOISE_DETAIL_RANGE	(1.0, 5.0)
BAG_NOISE_ROUGHNESS_RANGE	(0.2, 0.7)
BAG_COLOR_MIX_RANGE	(0.3, 0.8)
SURFACE_NOISE_SCALE_RANGE	(4.0, 14.0)
SURFACE_NOISE_STRENGTH_RANGE	(0.05, 0.25)
TERRAIN_COLOR_NOISE_SCALE_RANGE	(1.5, 6.0)
TERRAIN_COLOR_NOISE_STRENGTH_RANGE	(0.1, 0.35)
Terrain & Debris	
FLOOR_NOISE_SCALE_RANGE	(1.5, 4.0)
FLOOR_DISPLACE_STRENGTH_RANGE	(0.5, 1.5)
DEBRIS_COUNT_RANGE	(80, 150)
DEBRIS_SIZE_RANGE	(0.05, 0.25)
DEBRIS_SIZE_MAX_LARGE	1.0
DEBRIS_SPREAD	4.0
DEBRIS_JAGGEDNESS_RANGE	(0.1, 0.8)
DEBRIS_USE_RIGIDBODIES	False
DEBRIS_EMBED_DEPTH_RANGE	(0.0, 0.05)
Color Randomization	
COLOR_SAT_MAX	0.4
COLOR_VALUE_RANGE	(0.15, 0.6)

The next step in image generation was a layer of post processing to emulate a deep sea, low light environment. In deep sea imagery, a common visual artifact is marine snow. These artifacts are caused by light reflecting off of very small floating objects and debris, creating the visual effect of blurry white dots. To account for this, we scatter some random white particles across a new canvas, apply a directional motion blur to mimic camera drift and finally alpha composite the canvas onto the



Figure 1: Unprocessed output from the underwater trashbag simulation.

input image. Another common artifact of deep sea imagery is a vignette effect, caused by the artificial light source creating a spotlight in the darkness. To account for this, we apply a radial gradient mask with a randomized center, falloff and stretch parameters, imitating the transition to darkness at the sides of the image. Finally, we also apply some gaussian sensor noise and a small blur to the final output to imitate camera imperfections. For bounding box annotation, we quickly leverage Blender’s built in world-to-camera-view() to compute the screen space positions of all vertices, then take the lexicographic min and max. A table showing a sample of the final synthetic training data is shown for reference (Figure 2).

5 Testing Synthetic Prewarm

To test the effect of pretraining on our simulation images, we perform:

1. A rough ablation study comparing the mean average precision (mAP) over epochs for real-only, synthetic-only and synthetic-prewarmed training runs and,
2. A comparison to see how mAP changes as the real-world training sample size decreases over real-only and synthetic-prewarmed runs.

Together, these tests should show meaningful information about the early training stage - if the prewarmed model has learned useful priors, we should expect the mAP to increase faster over epochs, especially at the beginning when the real-world training set has yet to be closely fit. Similarly, as the real-world training sample gets smaller, the prewarmed model should start to outperform the real-only model.

5.1 A Note on mAP

To calculate average precision, we start by evaluating the predictions across our dataset D . For each image $i \in \{1, \dots, |D|\}$, let $\hat{B}_i$ be the predicted bounding box and B_i^* be the ground truth bounding box. We begin by calculating the Intersection over Union (IoU) for the i -th image:

$$IoU(\hat{B}_i, B_i^*) = \frac{\text{Area}(\hat{B}_i \cap B_i^*)}{\text{Area}(\hat{B}_i \cup B_i^*)}$$

If an image contains no ground truth object ($B_i^* = \emptyset$) or the model makes no prediction ($\hat{B}_i = \emptyset$), the intersection area is zero, and therefore $IoU = 0$.

From here, since our model always makes a prediction, we arrange the $|D|$ predictions in a list L ranked from highest to lowest confidence. For rank k and IoU threshold τ , we then have a True Positive (TP) if $IoU_k \geq \tau$ and a False Positive (FP) otherwise. At this point, we can accumulate

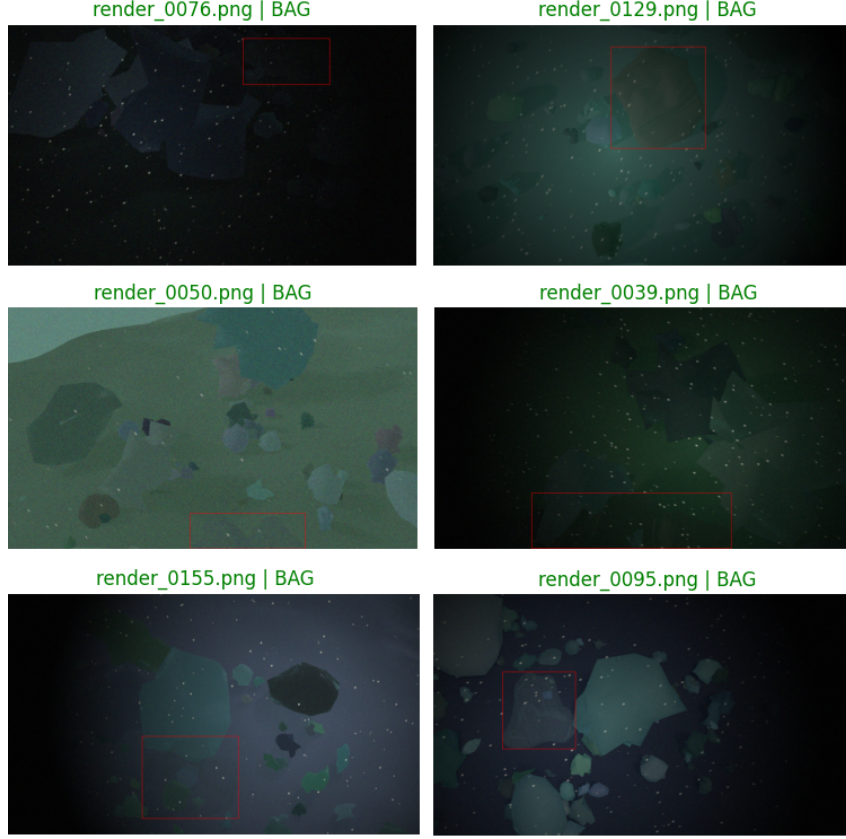


Figure 2: A sample of synthetic training data. Note the ground truth simulation annotations are readily available, illustrating the power of working in simulation - time intensive manual annotations are avoided.

the total TP and FP for each top- k subset $\{IoU_1, \dots, IoU_k\}$, for $k \in \{1, \dots, |D|\}$. Collecting this information for each subset lets us approximate the performance of the model across different confidence thresholds.

Let TP_k denote the accumulated True Positives at rank k . We define Precision (P_k) and Recall (R_k) for each rank as follows:

$$P_k = \frac{TP_k}{k}$$

$$R_k = \frac{TP_k}{M}$$

where M is the number of actual annotations in our dataset (images for which $B^* \neq \emptyset$). This yields a sequence of Precision-Recall pairs (R_k, P_k) that describe the model's performance for different confidence thresholds. These measurements describe a Precision-Recall curve for which AP is the area under it. Since the raw curve is jagged, an interpolated curve is used to compute AP in practice (for brevity, we leave the details to the reader). mAP is simply the average precision over a range of IoU thresholds τ , typically between 0.5 and 0.95.

5.2 Validation mAP over Epochs

In this section and the following section, all annotated real-world images of deep sea trash bags used for training were taken from the publicly available TrashCan dataset. Additionally, we use a sample size of 150 for the synthetic dataset, and the mAP variant used was $[0.5 : 0.95]$, also known as COCO.

For the first test, we:

1. Set up two datasets (real-only and synthetic-only).
2. Train three runs — real-only, synthetic-only, and a synthetic-prewarm then real finetune.
3. Plot validation mAP vs. epoch to compare the runs.

Each training run takes place over 30 epochs, with an image size of 416 and batch size of 16. The device used for training is Apple MPS, and the synthetic-prewarm run was initialized with the learned weights from the synthetic-only run. The results are shown in the following graph (Figure 3):

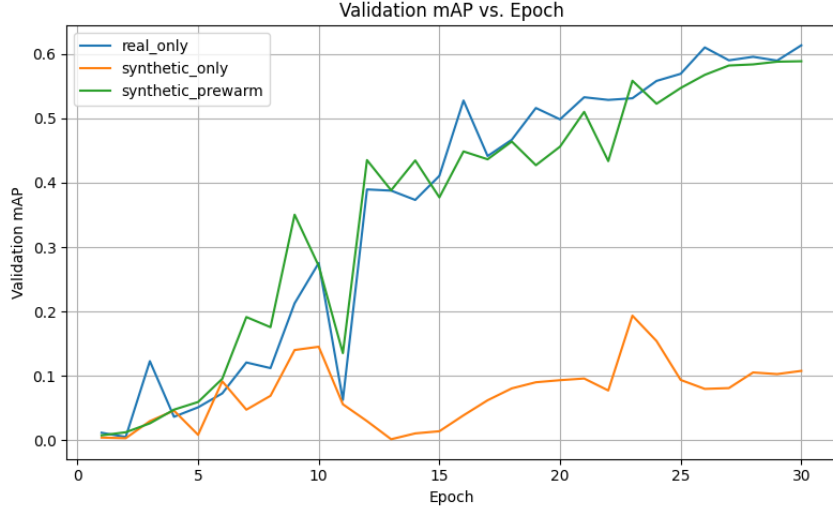


Figure 3: A graph of validation mAP over epochs for the three training runs.

Looking at the results, we can make a few quick observations. First, it appears that for up to epoch 15, the synthetic prewarm run has the highest mAP. After that, the real-only run performs better, perhaps due to the fact that TrashCan’s plastic bag dataset is large and the model is beginning to closely fit that target distribution. Overall, this graph is suggestive evidence that the hypothesis - that synthetic pretraining can help the model learn useful priors when real world data is scarce - holds true. We could infer that the simulated training data helps the model learn semantic information about the bag’s shape and appearance in the deep sea environment, even if its a loose approximation.

5.3 Final mAP vs. Downsampling Ratio

For the next test, we perform real-only and prewarmed training runs on decreasing real-world sample sizes. Without finding the total number of TrashCan images annotated with "plastic bag", we can quickly adjust the sample size by taking a ratio of the complete training data. Additionally, the training hyperparameters and hardware are the same as the previous test - 30 epochs, image size of 416 and batch size of 16. The results of this test are shown in Figure 4.

Surprisingly, the prewarm run only outperforms for 0.5x downsampling. A few potential reasons are:

1. The samples drawn might favor the real-only run. Repeated trials with multiple samples are needed to construct an accurate performance estimate.
2. More epochs may be needed to correct the sim2real gap between the synthetic prior and real target distribution - perhaps once these deficiencies are corrected, the prewarmed model can more effectively leverage the semantic information contained in the prior.

Overall, the quick spike in the prewarm run is suggestive evidence that the model has learned useful information from pretraining, especially in the context of the previous test. While more experimentation and potential refinements to the simulation are needed to fully assess the benefit of synthetic prewarm for underwater plastic bag detection, these initial results are promising and invite further research.

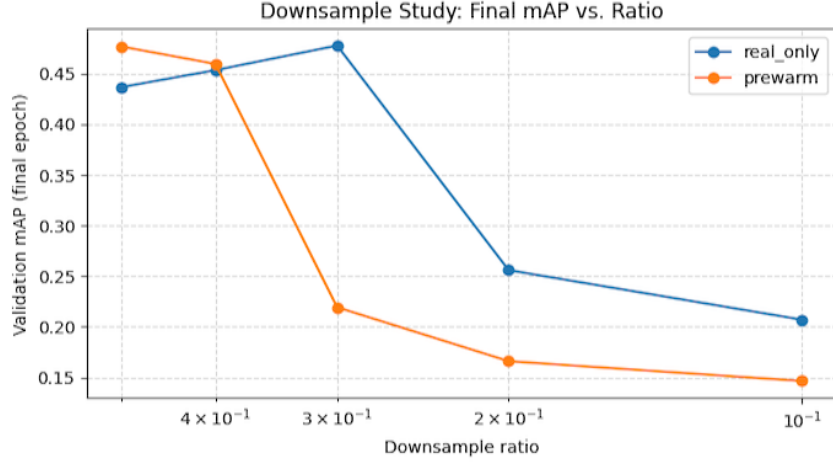


Figure 4: A graph of validation mAP over epochs for the three training runs.

6 References

References

- [1] Karin F. Kvale Shiye Zhao et al. *The distribution of subsurface microplastics in the ocean*. Nature. Accessed: 6 May 2026. URL: <https://www.nature.com/articles/s41586-025-08818-1>.
- [2] Ross Girshick et al. *Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation*. arXiv. Accessed: 10 May 2026. URL: <https://arxiv.org/abs/1311.2524>.
- [3] Xiang Li et al. *Generalized Focal Loss: Learning Qualified Predictive Bounding Boxes for Object Detection*. arXiv. Accessed: 10 May 2026. URL: <https://arxiv.org/abs/2006.04388>.
- [4] Tsung-Yi Lin et al. *Microsoft COCO: Common Objects in Context*. arXiv. Accessed: 10 May 2026. URL: <https://arxiv.org/abs/1405.0312>.
- [5] Joseph Redmon et al. *You Only Look Once: Unified, Real-Time Object Detection*. arXiv. Accessed: 10 May 2026. URL: <https://arxiv.org/abs/1506.02640>.
- [6] Zhaohui Zheng et al. *Distance-IoU Loss: Faster and Better Learning for Bounding Box Regression*. arXiv. Accessed: 10 May 2026. URL: <https://arxiv.org/abs/1911.08287>.